File Handling & I/O Operations in C#

🔹 1. What is File Handling?

File Handling means:
👉 Creating, reading, writing, appending, and deleting files on disk.

In C#, this is mainly done using the **System.IO** namespace.

🔹 2. What is I/O (Input / Output)?

- **Input** → Reading data (from file, user, network)
- **Output** → Writing data (to file, console, etc.)

👉 In file handling:

- Reading = Input
- Writing = Output

🔹 3. System.IO Namespace (Core of File Handling)

Key classes:

Class	Purpose
File	Work with files (static methods)
FileInfo	File details + instance-based
Directory	Work with folders
DirectoryInfo	Instance-based directory handling
FileStream	Low-level byte operations
StreamReader	Read text files
StreamWriter	Write text files

Class	Purpose
BinaryReader / BinaryWriter	Work with binary data

🔹 **4. File Class (Most Common)**

✅ **Write to File**

```
File.WriteAllText("data.txt", "Hello World");
```

✅ **Read from File**

```
string content = File.ReadAllText("data.txt");

Console.WriteLine(content);
```

✅ **Append Text**

```
File.AppendAllText("data.txt", "\nNew Line Added");
```

✅ **Check if File Exists**

```
if (File.Exists("data.txt"))

{

   Console.WriteLine("File exists");

}
```

🔹 **5. FileInfo (Object-Oriented Way)**

```
FileInfo file = new FileInfo("data.txt");


file.Create();

file.WriteAllText("Hello from FileInfo");
```

👉 Difference:

- File → Static methods (quick operations)
- FileInfo → More control, reusable object

🔹 6. Stream Concept (Very Important)

👉 Everything in I/O is based on **Streams**

A **Stream** = flow of data (like water in a pipe)

Types:

- FileStream
- MemoryStream
- NetworkStream

🔹 7. FileStream (Low-Level Control)

Example: Writing bytes

```
using (FileStream fs = new FileStream("data.bin", FileMode.Create))
{
    byte[] data = Encoding.UTF8.GetBytes("Hello");
    fs.Write(data, 0, data.Length);
}
```

Example: Reading bytes

```
using (FileStream fs = new FileStream("data.bin", FileMode.Open))
{
    byte[] buffer = new byte[fs.Length];
    fs.Read(buffer, 0, buffer.Length);
    Console.WriteLine(Encoding.UTF8.GetString(buffer));
}
```

👉 Used when:

- Large files
- Binary data

- Performance-critical apps

🔹 8. StreamReader & StreamWriter (Text Files)

✅ **Writing**

```
using (StreamWriter writer = new StreamWriter("data.txt"))
{
    writer.WriteLine("Hello");
    writer.WriteLine("C# File Handling");
}
```

✅ **Reading**

```
using (StreamReader reader = new StreamReader("data.txt"))
{
    string line;
    while ((line = reader.ReadLine()) != null)
    {
        Console.WriteLine(line);
    }
}
```

👉 Best for:

- Logs
- Config files
- Text processing

🔹 9. File Modes (Important for Interviews)

```
FileStream fs = new FileStream("data.txt", FileMode.Create);
```

Mode	Meaning
Create	Create new (overwrite if exists)
Open	Open existing
Append	Add to file
CreateNew	Error if file exists
OpenOrCreate	Open or create

🔹 10. Working with Directories

```
Directory.CreateDirectory("MyFolder");


string[] files = Directory.GetFiles("MyFolder");


foreach (var file in files)
{
    Console.WriteLine(file);
}
```

🔹 11. Exception Handling in File I/O ⚠️

Always handle exceptions:

```
try
{
    string data = File.ReadAllText("data.txt");
}
catch (FileNotFoundException)
{
```

```
    Console.WriteLine("File not found");
}
catch (IOException ex)
{
    Console.WriteLine(ex.Message);
}
```

🔹 12. Using Statement (VERY IMPORTANT)

```
using (StreamWriter writer = new StreamWriter("file.txt"))
{
    writer.WriteLine("Hello");
}
```

👉 Why?

- Automatically closes file
- Prevents memory leaks
- Ensures file is not locked

🔹 13. Binary File Handling

```
using (BinaryWriter bw = new BinaryWriter(File.Open("data.bin", FileMode.Create)))
{
    bw.Write(100);
    bw.Write("Hello");
}
using (BinaryReader br = new BinaryReader(File.Open("data.bin", FileMode.Open)))
{
    int num = br.ReadInt32();
```

```
    string text = br.ReadString();
}
```

🔹 14. Real-Time Use Cases

💼 Where you use file handling:

- Logging systems
- Configuration files (appsettings.json)
- Data export/import (CSV, Excel)

🔹 15. Best Practices 🚀

✓ Always use using
✓ Handle exceptions
✓ Avoid loading huge files into memory
✓ Use async methods for better performance
✓ Validate file paths

🔹 16. Async File Handling (Modern C#)

```
await File.WriteAllTextAsync("data.txt", "Hello Async");


string content = await File.ReadAllTextAsync("data.txt");
```

👉 Improves performance in:

- Web APIs
- UI apps